Assignment 5 — Using Move Evaluations from Previous Iterations in Local Search

Jędrzej Pluta 160327 Filip Podalak 160311

1 Problem description

We are given $n = 200$ nodes, each with x and y coordinates and a cost. The task is to select exactly half of the nodes (rounded up, so 100) and form a Hamiltonian cycle through them, minimising the total length of the cycle plus the total cost of the selected nodes:

$$f(C) = \sum_{(i,j)\in C} d(i,j) + \sum_{i\in C} c(i).$$

Distances are Euclidean distances rounded to the nearest integer and are computed once into a distance matrix right after reading an instance. All algorithms use only the distance matrix and the node costs. There are two instances, TSPA and TSPB.

This assignment improves the time efficiency of the *steepest* local search by re-using move evaluations (deltas) from previous iterations, stored in a list of improving moves *LM*. The neighbourhood is the best one from Assignment 3: 2-opt (two-edges exchange) inside the cycle plus the exchange of a selected node with an unselected one. It contains 10 000 node exchanges and 4 850 2-opt moves, 14 850 moves in total. The mechanism is used separately from the candidate moves of Assignment 4, and, as the optional extension, together with them.

2 Implemented algorithms

2.1 Moves and their deltas

All moves are stored by *node ids*, not by positions, because positions change whenever a segment is reversed.

- **2-opt**: remove edges (a, b) and (c, d), add (a, c) and (b, d), $\Delta = d(a,c)+d(b,d)-d(a,b)-d(c,d)$.
- **Node exchange**: the selected node x between p and q is replaced by the unselected node v, $\Delta = d(p,v) + d(v,q) - d(p,x) - d(x,q) + c(v) - c(x)$.

The delta of a move depends only on the nodes named in the move, so it stays correct for as long as the removed edges exist in the solution (and, for an exchange, v is unselected).

2.2 Applicability of a stored move

When browsing *LM* from the best move, a stored move falls into one of three situations:

1. **A removed edge no longer exists** (at least one of them): the move is deleted from *LM*.

2. **Both edges exist, but in a different relative direction than when the move was saved** (e.g. $a \to b$ but $d \to c$ for a 2-opt move): the move cannot be applied now, but it may become applicable after a later segment reversal. It stays in LM and browsing continues with the next move.

3. **Both edges exist in the same relative direction** (both as saved, or both reversed): the move is applied and removed from LM.

For a 2-opt move with $b = \text{succ}(a)$, $d = \text{succ}(c)$ the move is applied by reversing the path $b \dots c$. If both edges are reversed ($a = \text{succ}(b)$, $c = \text{succ}(d)$) the path $a \dots d$ is reversed instead; the added edges are the same, (a, c) and (b, d). For a node exchange the nodes p, x, q must still be consecutive (in either direction) and v must still be unselected; the two edges of an exchange can never be in mixed directions, so situation 2 only occurs for 2-opt moves.

2.3 Evaluating new moves

After a move is applied only moves that involve a *new* edge, or a node whose status changed, have a delta that is not already in LM. Only those are evaluated:

- **After a 2-opt move** the new edges are (a, c) and (b, d). Every new edge is paired with every other edge of the cycle (a pair of two new edges is evaluated once). For the four endpoints a, b, c, d, which got a new neighbour, all exchanges with all unselected nodes are evaluated.
- **After a node exchange** (x replaced by v between p and q) the new edges are (p, v) and (v, q). Both are paired with all other edges for 2-opt. The nodes p, v, q are exchanged with all unselected nodes, and the node x, which has just become unselected, is combined with every other selected node.

Relatively inverted edges. For every pair of edges (a, b), (c, d) that is considered there are two reconnections: $\{(a, c), (b, d)\}$, which is applicable in the current orientation, and $\{(a, d), (b, c)\}$, which would split the cycle now but becomes a valid move if one of the two edges is later reversed relative to the other. Both are evaluated, and both are stored if improving; the second one is stored as the move (a, b, d, c) and waits in LM in situation 2. Without this the list would miss improving moves, and an empty LM would not imply a local optimum. The same holds for the initial solution: all $\binom{100}{2} - 100 = 4\,850$ edge pairs are evaluated in both reconnections, together with all $10\,000$ exchanges.

2.4 Steepest local search with the list of moves

LM is a multiset ordered by Δ (best first). Because it contains *all* currently known improving moves, the first applicable move is the best move of the whole neighbourhood, so the search follows the same steepest rule as the baseline, but without re-scanning the neighbourhood.

Algorithm 1 Steepest local search with a list of improving moves

```
1: x ← random solution;   LM ← ∅
2: evaluate all moves applicable to x, including relatively inverted 2-opt moves; insert the improving ones into LM
3: repeat
4:     for m ∈ LM from the best do
5:         if a removed edge of m does not exist, or the exchanged node has changed then
6:             delete m from LM
7:         else if the edges of m have a different relative direction then
8:             continue                                                    ▷ keep m
9:         else
10:            delete m from LM;   x ← m(x);   update positions
11:            evaluate the new moves (new edges, changed nodes) and insert the improving ones into LM
12:            break
13:        end if
14:    end for
15: until no move was applied (the whole LM was browsed)
```

A move can end up in LM twice (it was kept as stale, then regenerated after its edges were created again). This is harmless: after the first copy is applied, the second one is no longer applicable and is deleted on the next browse.

2.5 Combination with candidate moves (optional)

With candidate lists ($K = 10$, the K nodes v with the smallest $d(u, v) + c(v)$) a move enters LM only if it adds at least one candidate edge, exactly as in Assignment 4. This applies to the initial evaluation, to the newly generated moves, and to the inverted 2-opt moves. The filter is a constant-time lookup in a 0/1 matrix. The search for new moves still scans the edges of the cycle, so the saving comes from fewer evaluated deltas and a shorter list, not from fewer loop iterations.

2.6 Baseline and earlier methods

The baseline is the Assignment 3 steepest local search with the same neighbourhood, evaluated in full in every iteration, without any of the mechanisms above. All methods start from the same 200 random solutions per instance. The tables also include all methods from Assignments 1–4, run again in the same program so that running times are comparable. Each method was run 200 times; constructive methods start from every node once.

3 Results: objective function

Table 1: Objective function: average (minimum–maximum) over 200 runs. The best average per instance is in bold.

Group	Method	TSPA	TSPB
A3	LS steepest, node swap, random start	89 885 (82 411–99 005)	65 963 (59 709–72 611)
A3	LS steepest, 2-opt, random start **(baseline)**	76 380 (74 031–79 775)	**52 674 (50 200–56 190)**
A3	LS greedy, node swap, random start	88 237 (81 707–95 233)	63 922 (56 525–68 774)
A3	LS greedy, 2-opt, random start	76 140 (73 676–78 377)	52 979 (50 628–57 101)
A3	LS steepest, node swap, greedy start	73 472 (72 415–74 736)	59 233 (55 406–63 516)
A3	LS steepest, 2-opt, greedy start	73 179 (72 088–74 544)	58 666 (54 818–63 178)
A3	LS greedy, node swap, greedy start	73 473 (72 415–74 736)	59 233 (55 406–63 516)
A3	LS greedy, 2-opt, greedy start	**73 173 (72 088–74 544)**	58 661 (54 756–63 269)
A4	LS steepest, 2-opt, candidate moves (K = 10)	80 064 (76 341–86 028)	52 798 (50 192–56 956)
A5	LS steepest, 2-opt, list of moves (LM)	76 381 (74 089–79 775)	52 675 (50 200–56 220)
A5	LS steepest, 2-opt, list of moves + candidates (K = 10)	80 099 (76 341–86 028)	52 776 (50 192–56 956)

4 Results: running time

All methods were run in one program on the same machine (C++17, `-O2`). Times include building the starting solution, except for the shared random starts, whose generation is the Random row. The candidate lists are computed once per instance and are not counted.

Table 2: Running time per run in milliseconds: average (minimum–maximum) over 200 runs.

Group	Method	TSPA	TSPB
A3	LS steepest, node swap, random start	7.41 (5.81–9.91)	7.33 (6.03–9.64)
A3	LS steepest, 2-opt, random start **(baseline)**	5.01 (4.32–19.7)	4.96 (4.35–5.70)
A3	LS greedy, node swap, random start	1.64 (0.873–3.58)	1.57 (0.852–2.97)
A3	LS greedy, 2-opt, random start	1.09 (0.702–1.58)	1.06 (0.681–1.62)
A3	LS steepest, node swap, greedy start	1.09 (0.868–1.71)	1.24 (0.911–4.97)
A3	LS steepest, 2-opt, greedy start	1.08 (0.889–1.45)	1.26 (0.972–1.90)
A3	LS greedy, node swap, greedy start	1.25 (1.000–1.79)	1.33 (1.02–2.13)
A3	LS greedy, 2-opt, greedy start	1.33 (1.02–2.00)	1.41 (1.08–2.10)
A4	LS steepest, 2-opt, candidate moves (K = 10)	1.52 (1.32–2.06)	1.60 (1.32–2.13)
A5	LS steepest, 2-opt, list of moves (LM)	2.17 (1.75–3.54)	1.88 (1.50–2.56)
A5	LS steepest, 2-opt, list of moves + candidates (K = 10)	**0.889 (0.758–1.87)**	**0.842 (0.729–1.31)**

4.1 Evaluated moves

Table 3 shows the average number of evaluated moves (deltas) and applied moves per run for the steepest searches with random starts. For the baseline an iteration evaluates all 14 850 moves, including the final one that finds no improvement.

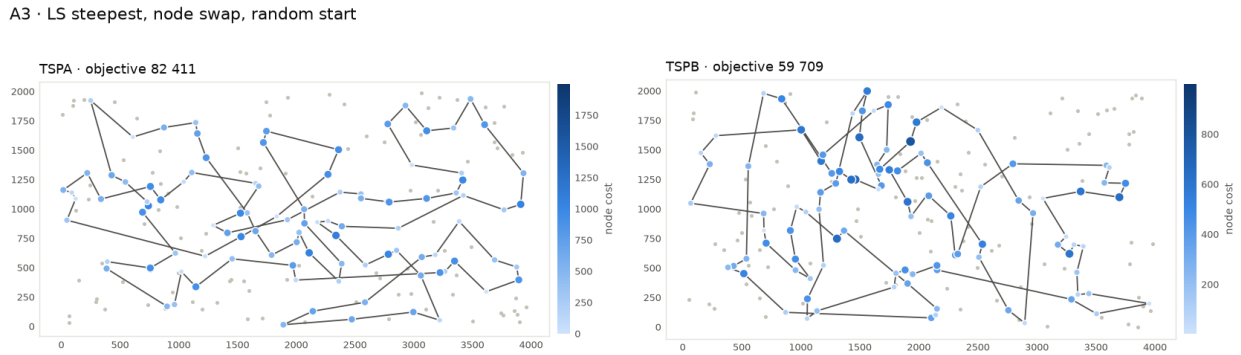
Table 3: Average number of evaluated and applied moves and average time per run (200 runs, same random starts).

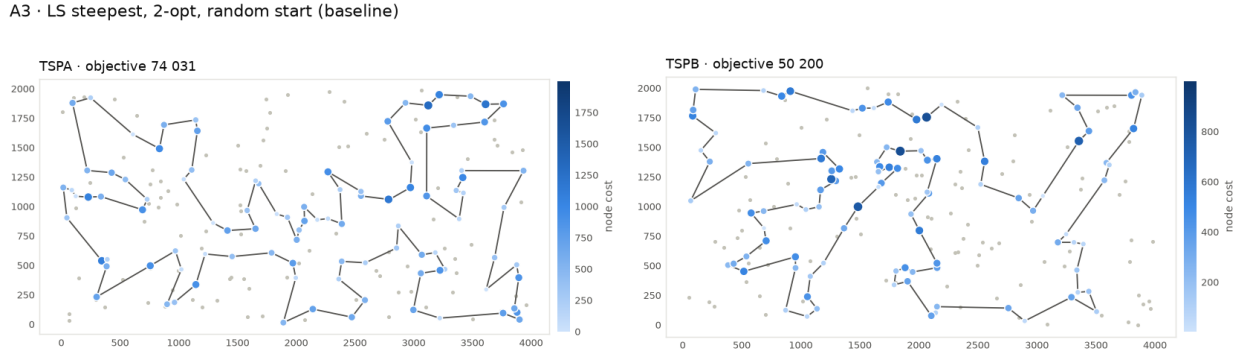
	TSPA			TSPB		
Method	evaluated	applied	time [ms]	evaluated	applied	time [ms]
LS steepest, 2-opt, random start (baseline)	1 972 526	131.8	5.01	1 991 311	133.1	4.96
LS steepest, 2-opt, candidate moves (K = 10)	226 889	125.0	1.52	233 276	131.2	1.60
LS steepest, 2-opt, list of moves (LM)	123 246	131.8	2.17	124 112	132.9	1.88
LS steepest, 2-opt, list of moves + candidates (K = 10)	14 166	124.9	0.889	12 909	131.5	0.842

5 Visualisation of the best solutions

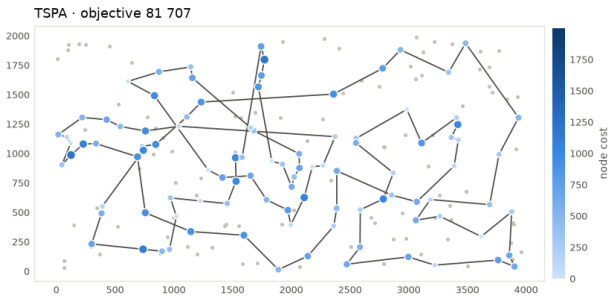
Each figure shows the best solution of one method on TSPA (left) and TSPB (right). Selected nodes are coloured and sized by their cost, with darker and larger meaning more expensive. Unselected nodes are small grey dots, and the line is the cycle, including the closing edge.

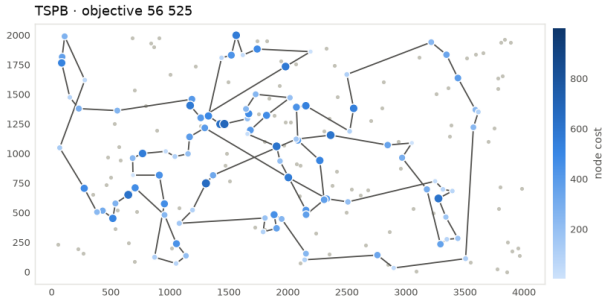
Assignment 3



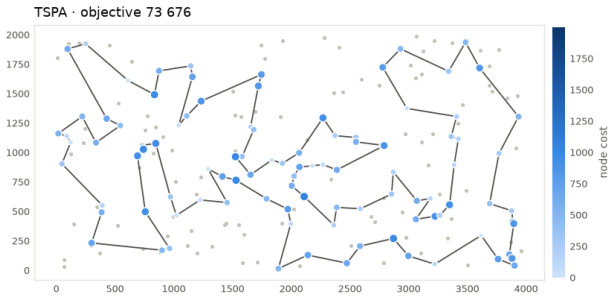


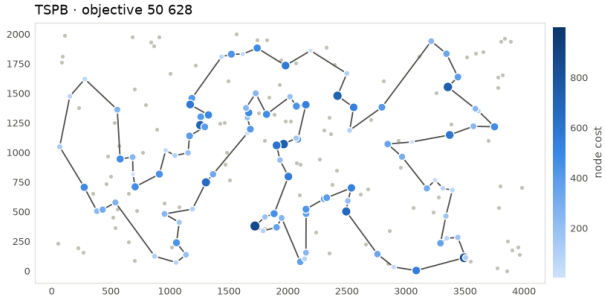
A3 · LS greedy, node swap, random start



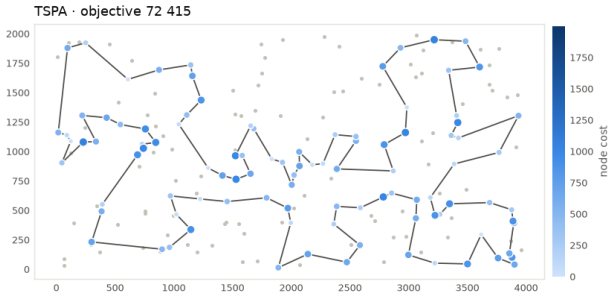


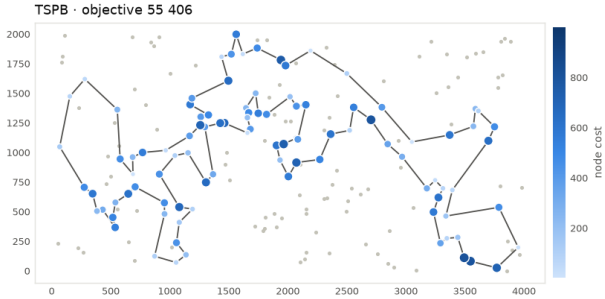
A3 · LS greedy, 2-opt, random start



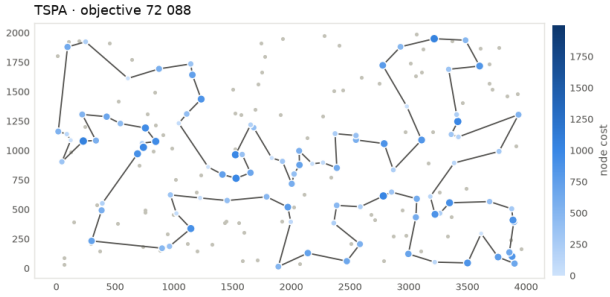


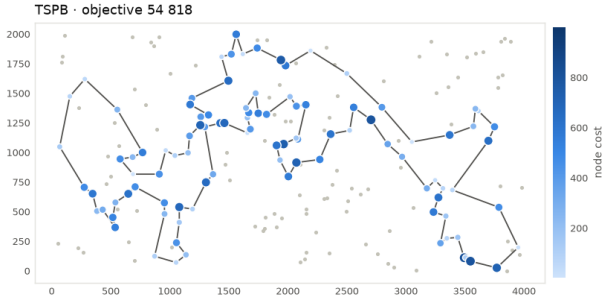
A3 · LS steepest, node swap, greedy start



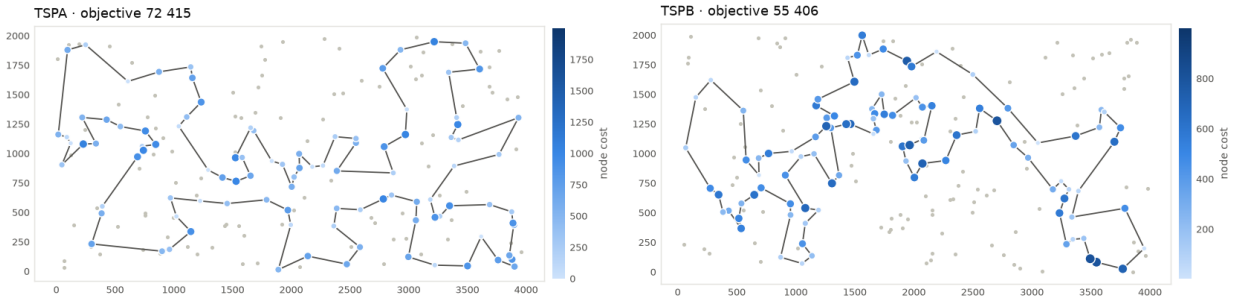


A3 · LS steepest, 2-opt, greedy start

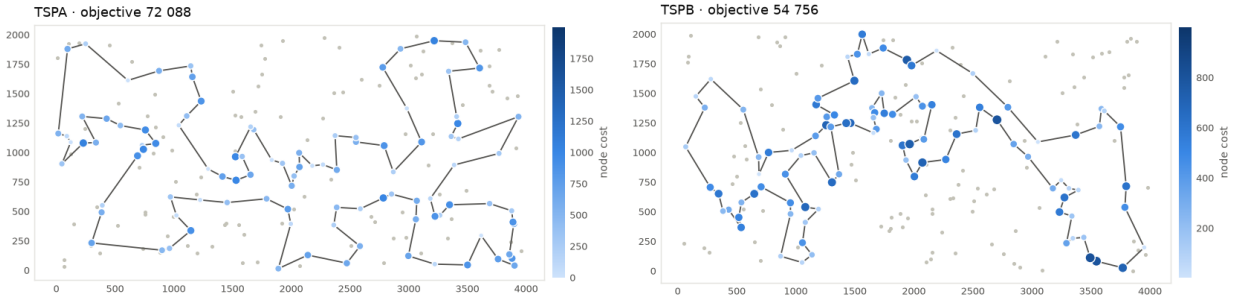




A3 · LS greedy, node swap, greedy start

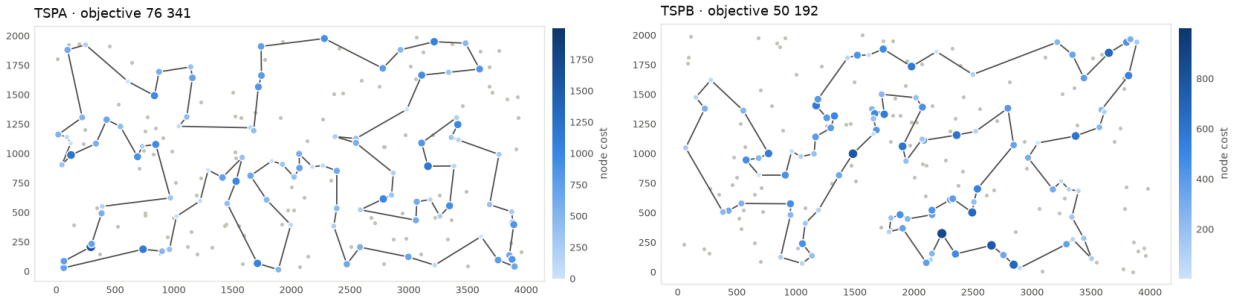


A3 · LS greedy, 2-opt, greedy start



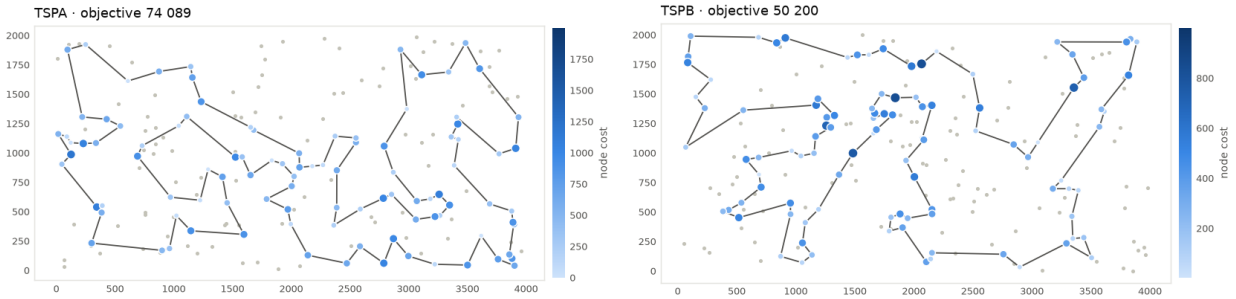
Assignment 4

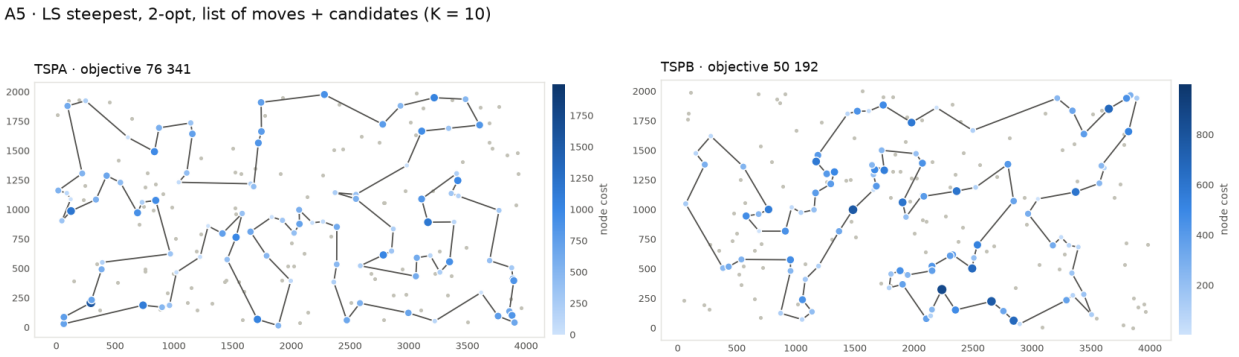
A4 · LS steepest, 2-opt, candidate moves (K = 10)



Assignment 5

A5 · LS steepest, 2-opt, list of moves (LM)





6 Best solutions (node indices, starting from 0)

Each list is the best cycle a method found, in visiting order. The cycle closes from the last node back to the first. Methods are in the same order as in the tables.

TSPA

A3 · LS steepest, node swap, random start (82 411) 177, 99, 134, 72, 176, 151, 57, 182, 192, 114, 181, 158, 150, 106, 93, 130, 159, 170, 107, 113, 89, 42, 135, 73, 87, 189, 187, 2, 171, 127, 97, 161, 30, 160, 83, 123, 152, 108, 48, 80, 91, 157, 71, 4, 35, 185, 45, 140, 53, 41, 58, 84, 179, 52, 139, 112, 62, 143, 67, 102, 141, 168, 63, 149, 116, 60, 49, 100, 90, 132, 76, 51, 16, 156, 180, 28, 144, 33, 9, 54, 194, 8, 85, 3, 169, 1, 164, 186, 136, 184, 178, 29, 101, 147, 37, 47, 50, 174, 131, 36

A3 · LS steepest, 2-opt, random start (baseline) (74 031) 85, 184, 178, 101, 55, 17, 29, 163, 24, 136, 186, 164, 142, 8, 194, 54, 9, 143, 33, 131, 36, 177, 52, 179, 53, 140, 144, 67, 16, 51, 35, 185, 28, 180, 132, 76, 71, 157, 91, 38, 80, 48, 31, 60, 108, 152, 123, 160, 49, 11, 63, 149, 116, 168, 95, 97, 127, 69, 161, 30, 187, 171, 2, 189, 141, 87, 45, 73, 135, 114, 181, 192, 158, 58, 84, 134, 176, 174, 150, 106, 89, 42, 151, 182, 57, 88, 78, 193, 153, 170, 159, 107, 130, 47, 147, 1, 169, 3, 50, 93

A3 · LS greedy, node swap, random start (81 707) 93, 50, 3, 169, 1, 85, 164, 9, 54, 174, 72, 134, 158, 192, 182, 57, 88, 78, 153, 159, 107, 89, 151, 106, 130, 47, 147, 29, 178, 136, 186, 62, 4, 76, 90, 123, 83, 97, 127, 69, 92, 171, 2, 187, 30, 141, 87, 102, 156, 35, 115, 5, 112, 139, 67, 144, 140, 53, 179, 52, 16, 51, 71, 157, 91, 80, 38, 180, 28, 185, 45, 73, 135, 41, 177, 36, 33, 132, 100, 152, 108, 60, 116, 149, 63, 168, 119, 11, 49, 160, 161, 189, 98, 114, 181, 58, 84, 131, 176, 150

A3 · LS greedy, 2-opt, random start (73 676) 50, 3, 1, 164, 186, 136, 178, 29, 101, 147, 47, 130, 159, 146, 170, 153, 193, 78, 88, 107, 57, 182, 138, 158, 192, 181, 114, 135, 73, 45, 102, 28, 180, 87, 141, 187, 30, 90, 123, 83, 160, 161, 2, 171, 69, 127, 97, 168, 63, 149, 116, 60, 49, 152, 108, 31, 48, 80, 38, 91, 157, 71, 132, 76, 4, 112, 139, 51, 16, 35, 156, 185, 67, 144, 52, 143, 33, 9, 54, 194, 8, 131, 36, 177, 179, 53, 140, 41, 58, 84, 134, 176, 174, 150, 106, 151, 42, 89, 113, 93

A3 · LS steepest, node swap, greedy start (72 415) 31, 116, 149, 63, 168, 11, 49, 60, 108, 152, 100, 90, 123, 83, 160, 97, 127, 69, 171, 2, 189, 187, 30, 141, 87, 45, 73, 135, 114, 181, 192, 158, 58, 84, 134, 72, 176, 150, 151, 182, 57, 64, 107, 88, 78, 193, 153, 170, 146, 159, 130, 113, 89, 42, 106, 93, 47, 147, 50, 3, 169, 1, 178, 101, 29, 163, 136, 186, 164, 142, 8, 174, 131, 54, 9, 33, 36, 177, 52, 179, 53, 140, 144, 67, 16, 51, 156, 35, 185, 102, 28, 180, 132, 76, 4, 71, 157, 91, 80, 48

A3 · LS steepest, 2-opt, greedy start (72 088) 31, 116, 149, 63, 168, 11, 49, 60, 108, 152, 100, 90, 123, 83, 160, 97, 127, 69, 171, 2, 189, 187, 30, 141, 87, 45, 73, 135, 114, 181, 192, 158, 58, 84, 134, 72, 176, 150, 151, 182, 57, 64, 107, 88, 78, 193, 153, 170, 146, 159, 130, 113, 89, 42, 106, 93, 47, 147, 50, 3, 169, 1,

178, 101, 29, 163, 136, 186, 164, 85, 174, 8, 54, 9, 33, 131, 36, 177, 52, 179, 53, 140, 144, 67, 16, 51, 156, 35, 185, 102, 28, 180, 132, 76, 4, 71, 157, 91, 80, 48

A3 · LS greedy, node swap, greedy start (72 415) 31, 116, 149, 63, 168, 11, 49, 60, 108, 152, 100, 90, 123, 83, 160, 97, 127, 69, 171, 2, 189, 187, 30, 141, 87, 45, 73, 135, 114, 181, 192, 158, 58, 84, 134, 72, 176, 150, 151, 182, 57, 64, 107, 88, 78, 193, 153, 170, 146, 159, 130, 113, 89, 42, 106, 93, 47, 147, 50, 3, 169, 1, 178, 101, 29, 163, 136, 186, 164, 142, 8, 174, 131, 54, 9, 33, 36, 177, 52, 179, 53, 140, 144, 67, 16, 51, 156, 35, 185, 102, 28, 180, 132, 76, 4, 71, 157, 91, 80, 48

A3 · LS greedy, 2-opt, greedy start (72 088) 31, 116, 149, 63, 168, 11, 49, 60, 108, 152, 100, 90, 123, 83, 160, 97, 127, 69, 171, 2, 189, 187, 30, 141, 87, 45, 73, 135, 114, 181, 192, 158, 58, 84, 134, 72, 176, 150, 151, 182, 57, 64, 107, 88, 78, 193, 153, 170, 146, 159, 130, 113, 89, 42, 106, 93, 47, 147, 50, 3, 169, 1, 178, 101, 29, 163, 136, 186, 164, 85, 174, 8, 54, 9, 33, 131, 36, 177, 52, 179, 53, 140, 144, 67, 16, 51, 156, 35, 185, 102, 28, 180, 132, 76, 4, 71, 157, 91, 80, 48

A4 · LS steepest, 2-opt, candidate moves (K = 10) (76 341) 169, 3, 50, 47, 130, 159, 146, 170, 153, 193, 78, 88, 107, 57, 182, 158, 192, 58, 84, 131, 36, 177, 52, 179, 53, 144, 67, 185, 45, 135, 114, 25, 87, 102, 35, 28, 180, 141, 187, 2, 171, 92, 104, 129, 94, 69, 127, 97, 30, 90, 123, 160, 152, 108, 49, 119, 168, 63, 149, 116, 60, 31, 48, 80, 38, 91, 157, 71, 76, 132, 51, 16, 139, 112, 115, 39, 186, 136, 163, 29, 101, 178, 184, 164, 9, 33, 54, 174, 176, 72, 134, 151, 150, 106, 89, 113, 93, 81, 85, 1

A5 · LS steepest, 2-opt, list of moves (LM) (74 089) 164, 8, 174, 150, 106, 43, 113, 89, 42, 151, 176, 72, 134, 58, 84, 131, 54, 9, 33, 36, 177, 179, 52, 16, 51, 4, 71, 157, 91, 80, 48, 31, 60, 108, 152, 49, 11, 63, 149, 116, 119, 168, 95, 97, 127, 69, 171, 2, 187, 189, 98, 87, 28, 180, 141, 30, 160, 123, 132, 76, 156, 35, 185, 67, 144, 53, 140, 45, 73, 135, 181, 192, 158, 111, 138, 182, 57, 64, 107, 88, 78, 193, 153, 170, 146, 159, 130, 93, 50, 3, 169, 1, 47, 37, 147, 101, 29, 178, 184, 136

A5 · LS steepest, 2-opt, list of moves + candidates (K = 10) (76 341) 50, 47, 130, 159, 146, 170, 153, 193, 78, 88, 107, 57, 182, 158, 192, 58, 84, 131, 36, 177, 52, 179, 53, 144, 67, 185, 45, 135, 114, 25, 87, 102, 35, 28, 180, 141, 187, 2, 171, 92, 104, 129, 94, 69, 127, 97, 30, 90, 123, 160, 152, 108, 49, 119, 168, 63, 149, 116, 60, 31, 48, 80, 38, 91, 157, 71, 76, 132, 51, 16, 139, 112, 115, 39, 186, 136, 163, 29, 101, 178, 184, 164, 9, 33, 54, 174, 176, 72, 134, 151, 150, 106, 89, 113, 93, 81, 85, 1, 169, 3

TSPB

A3 · LS steepest, node swap, random start (59 709) 66, 46, 72, 117, 8, 122, 151, 184, 3, 27, 34, 24, 76, 104, 196, 114, 10, 148, 143, 78, 54, 127, 80, 52, 129, 90, 153, 20, 14, 110, 2, 74, 181, 152, 167, 73, 59, 85, 45, 134, 157, 40, 47, 38, 21, 121, 58, 198, 63, 160, 69, 168, 189, 183, 30, 147, 116, 158, 6, 29, 132, 197, 175, 82, 87, 179, 51, 64, 33, 140, 154, 97, 155, 60, 71, 115, 170, 125, 65, 145, 28, 149, 50, 191, 130, 96, 48, 120, 133, 172, 136, 166, 156, 89, 57, 162, 192, 171, 99, 67

A3 · LS steepest, 2-opt, random start (baseline) (50 200) 124, 64, 40, 60, 157, 45, 134, 175, 71, 155, 197, 132, 29, 6, 115, 170, 177, 125, 154, 140, 47, 38, 21, 176, 30, 183, 130, 191, 158, 65, 145, 28, 149, 13, 146, 37, 116, 147, 42, 121, 198, 96, 48, 156, 32, 89, 57, 185, 76, 162, 192, 54, 18, 187, 70, 190, 81, 94, 105, 61, 196, 114, 10, 127, 80, 129, 90, 153, 20, 14, 2, 74, 171, 99, 179, 87, 82, 51, 85, 59, 73, 167, 181, 152, 98, 151, 3, 184, 86, 46, 72, 92, 120, 133, 136, 172, 8, 117, 160, 69

A3 · LS greedy, node swap, random start (56 525) 67, 24, 98, 21, 38, 47, 140, 154, 26, 125, 170, 97, 155, 175, 134, 45, 157, 71, 115, 180, 6, 197, 29, 132, 178, 65, 149, 37, 146, 13, 28, 145, 158, 130, 191, 30, 156, 89, 48, 96, 58, 198, 121, 183, 189, 168, 172, 136, 133, 120, 72, 117, 160, 69, 122, 151, 152, 181, 34, 27, 3, 46, 86, 76, 185, 57, 190, 70, 187, 196, 114, 10, 2, 171, 99, 87, 179, 167, 73, 51, 85, 59, 60, 40, 33, 64, 184, 135, 162, 54, 192, 80, 74, 20, 14, 153, 52, 90, 129, 127

A3 · LS greedy, 2-opt, random start (50 628) 31, 99, 171, 17, 188, 2, 14, 20, 74, 153, 90, 129, 127, 80, 192, 162, 54, 78, 10, 148, 114, 196, 18, 187, 70, 190, 104, 76, 185, 79, 57, 89, 156, 48, 96, 198, 121, 130, 191, 183, 30, 176, 21, 38, 47, 140, 154, 97, 115, 170, 125, 177, 158, 149, 28, 65, 178, 132, 29, 6, 175, 134, 45, 157, 60, 71, 40, 33, 64, 69, 160, 172, 136, 133, 120, 117, 72, 46, 86, 3, 184, 173, 122, 151, 98, 73, 59, 107, 85, 51, 167, 82, 87, 179, 181, 152, 34, 24, 66, 67

A3 · LS steepest, node swap, greedy start (55 406) 30, 183, 191, 130, 63, 121, 198, 58, 96, 48, 43, 156, 89, 57, 104, 54, 78, 10, 196, 114, 148, 143, 90, 153, 118, 110, 93, 9, 2, 188, 14, 20, 74, 182, 52, 129, 127, 80, 192, 162, 144, 185, 76, 135, 27, 111, 98, 151, 122, 173, 184, 3, 86, 46, 72, 117, 8, 120, 133, 136, 172, 160, 69, 168, 189, 21, 64, 33, 47, 140, 97, 141, 40, 60, 157, 45, 134, 175, 71, 155, 115, 180, 6, 197, 36, 29, 132, 103, 178, 65, 28, 149, 158, 177, 170, 125, 26, 154, 38, 176

A3 · LS steepest, 2-opt, greedy start (54 818) 158, 149, 28, 65, 178, 103, 132, 29, 36, 197, 6, 180, 115, 155, 71, 175, 134, 45, 157, 60, 141, 40, 33, 64, 21, 189, 168, 69, 160, 172, 136, 133, 120, 8, 117, 72, 46, 86, 3, 184, 173, 122, 151, 98, 111, 27, 135, 76, 185, 144, 162, 192, 80, 127, 129, 52, 182, 153, 74, 20, 14, 188, 2, 9, 93, 110, 118, 90, 143, 148, 114, 196, 10, 78, 54, 104, 57, 89, 156, 43, 48, 96, 58, 198, 121, 63, 130, 191, 183, 30, 176, 38, 47, 140, 154, 97, 170, 177, 125, 26

A3 · LS greedy, node swap, greedy start (55 406) 30, 183, 191, 130, 63, 121, 198, 58, 96, 48, 43, 156, 89, 57, 104, 54, 78, 10, 196, 114, 148, 143, 90, 153, 118, 110, 93, 9, 2, 188, 14, 20, 74, 182, 52, 129, 127, 80, 192, 162, 144, 185, 76, 135, 27, 111, 98, 151, 122, 173, 184, 3, 86, 46, 72, 117, 8, 120, 133, 136, 172, 160, 69, 168, 189, 21, 64, 33, 47, 140, 97, 141, 40, 60, 157, 45, 134, 175, 71, 155, 115, 180, 6, 197, 36, 29, 132, 103, 178, 65, 28, 149, 158, 177, 170, 125, 26, 154, 38, 176

A3 · LS greedy, 2-opt, greedy start (54 756) 170, 26, 154, 38, 176, 30, 183, 191, 130, 63, 121, 198, 58, 96, 48, 43, 156, 89, 57, 104, 54, 78, 10, 196, 114, 148, 56, 118, 110, 93, 9, 2, 188, 14, 20, 74, 153, 182, 52, 90, 129, 127, 80, 192, 162, 144, 185, 76, 135, 27, 111, 98, 151, 122, 173, 184, 3, 86, 46, 72, 117, 8, 120, 133, 136, 172, 160, 69, 168, 189, 21, 64, 33, 47, 140, 97, 141, 40, 60, 157, 45, 134, 175, 71, 155, 115, 180, 6, 197, 36, 29, 132, 103, 178, 65, 145, 28, 149, 158, 125

A4 · LS steepest, 2-opt, candidate moves (K = 10) (50 192) 187, 169, 81, 94, 105, 61, 196, 114, 10, 78, 54, 192, 80, 127, 129, 90, 153, 14, 2, 20, 74, 171, 39, 99, 163, 88, 126, 179, 87, 82, 51, 85, 59, 73, 167, 181, 152, 34, 24, 31, 67, 66, 162, 104, 76, 135, 3, 184, 151, 122, 46, 72, 120, 8, 133, 136, 172, 160, 69, 124, 64, 40, 60, 157, 45, 134, 175, 71, 155, 6, 29, 132, 65, 145, 28, 149, 158, 26, 125, 177, 170, 97, 154, 140, 47, 38, 21, 183, 30, 191, 130, 121, 198, 96, 48, 156, 89, 57, 190, 70

A5 · LS steepest, 2-opt, list of moves (LM) (50 200) 167, 73, 59, 85, 51, 82, 87, 179, 99, 171, 74, 2, 14, 20, 153, 90, 129, 80, 127, 10, 114, 196, 61, 105, 94, 81, 190, 70, 187, 18, 54, 192, 162, 76, 185, 57, 89, 32, 156, 48, 96, 198, 121, 42, 147, 116, 37, 146, 13, 149, 28, 145, 65, 158, 191, 130, 183, 30, 176, 21, 38, 47, 140, 154, 125, 177, 170, 115, 6, 29, 132, 197, 155, 71, 175, 134, 45, 157, 60, 40, 64, 124, 69, 160, 117, 8, 172, 136, 133, 120, 92, 72, 46, 86, 184, 3, 151, 98, 152, 181

A5 · LS steepest, 2-opt, list of moves + candidates (K = 10) (50 192) 39, 171, 74, 20, 2, 14, 153, 90, 129, 127, 80, 192, 54, 78, 10, 114, 196, 61, 105, 94, 81, 169, 187, 70, 190, 57, 89, 156, 48, 96, 198, 121, 130, 191, 30, 183, 21, 38, 47, 140, 154, 97, 170, 177, 125, 26, 158, 149, 28, 145, 65, 132, 29, 6, 155, 71, 175, 134, 45, 157, 60, 40, 64, 124, 69, 160, 172, 136, 133, 8, 120, 72, 46, 122, 151, 184, 3, 135, 76, 104, 162, 66, 67, 31, 24, 34, 152, 181, 167, 73, 59, 85, 51, 82, 87, 179, 126, 88, 163, 99

7 Source code

The code is at `github.com/fpodalak/Evolutionary-Computation/tree/main/Assignment5`. It is C++17 with no external libraries, plus Python script for the figures. To run it, put `TSPA.csv` and `TSPB.csv` in the folder:

```
make run
python3 plot.py
```

File	Contents
`src/LMSearch.cpp`	Assignment 5: steepest local search with the list of improving moves (`TSPSolver::local_search_lm`), with and without candidate moves
`src/TSPSolver.h`, `src/TSPSolver.cpp`	Assignment 3 and 4 solver (construction heuristics, local search, candidate moves), extended by move counters and a candidate lookup matrix
`src/Types.h`	move types, including the node-based `ListMove`
`src/instance.h`, `src/construct.h`	Assignment 2 regret construction, reused unchanged
`src/main.cpp`	the experiment: all methods, 200 timed runs each
`plot.py`	figures

8 Conclusions

Speed. Re-using move evaluations makes the steepest local search with 2-opt and node exchange 2.3× faster on TSPA (2.17 ms vs 5.01 ms per run) and 2.6× faster on TSPB (1.88 ms vs 4.96 ms) than the baseline. The number of evaluated moves drops from 1 972 526 to 123 246 per run on TSPA, a factor of 16, and from 1 991 311 to 124 112 on TSPB (factor 16). The running time falls less than the number of evaluations, because an *LM* iteration is more expensive per evaluated move than a baseline iteration: it also maintains a sorted multiset, browses stale or inverted moves, scans the cycle for pairs with the new edges, and updates positions after every reversal. The baseline spends about 3 ns per evaluated move on TSPA, and the list-based search about 18 ns.

Quality. The list-based search uses the same neighbourhood and the same steepest rule as the baseline, and the list always contains the best improving move (checked by the tests). The only difference is the choice among moves with the same delta. The average objective is therefore practically unchanged: 76 381 vs 76 380 on TSPA (+0.00%) and 52 675 vs 52 674 on TSPB (+0.00%).

Comparison with candidate moves. Candidate moves (Assignment 4) are 3.3× faster than the baseline on TSPA, but change the neighbourhood and cost +4.8% in the average objective on TSPA (+0.2% on TSPB). The list of moves is 1.4× slower than candidate moves on TSPA (1.2× slower on TSPB) and loses nothing in quality, because it only avoids repeating evaluations. The two ideas reduce different costs.

Combination. List of moves together with candidate moves evaluates only 14 166 moves per run on TSPA (12 909 on TSPB), a factor of 16 (18) fewer than candidate moves alone. It is 5.6× faster than the baseline on TSPA (5.9× faster on TSPB) and 2.4× faster than the list of moves alone on TSPA. The average objective is 80 099 on TSPA and 52 776 on TSPB, which differs from candidate moves alone by +0.04% and -0.04%, since the neighbourhood is the same.